

Woche 6: Weitergehende Programmierung und Programmiertechniken

Einleitung

In dieser Woche liegt der Fokus auf dem Testen von Anwendungen und Dokumentation - Elemente, die für jeden Programmierer unverzichtbar sind.

Dabei sind die Lernmaterialien und Aufgaben darauf ausgerichtet, praktische Fähigkeiten in den Bereichen Code-Test und Dokumentation zu vermitteln.

Es wird die Bedeutung des Testens von Anwendungen beleuchtet und erlernt, wie man effektive Unit-Tests mit den "unittest" Modulen erstellt. Diese Fähigkeiten sind entscheidend, um die Qualität der Softwareentwicklung zu gewährleisten und sicherzustellen, dass alle Teile des Codes wie erwartet funktionieren.

Schließlich wird das Thema Dokumentation behandelt - ein oft übersehener, aber dennoch entscheidender Aspekt der Softwareentwicklung. Eine gut dokumentierte Anwendung erleichtert anderen Entwicklern (und auch Ihnen selbst in der Zukunft) das Verständnis und die Wartung des Codes.

Gesamtüberblick

Hier ein Überblick über die Inhalte und Aktivitäten der aktuellen Woche:

- Selbststudium:
 - Testen von Anwendungen
 - Unittest
 - Dokumentation von Anwendungen
- Aufgabe:
 - Unittests anlegen
 - Testbaren Code entwerfen
 - Eine Anwendung dokumentieren
- Tag 6:
 - Vertiefung: Tests entwerfen
 - Ausblick auf die Folgewoche

Inhalte und thematische Abgrenzung

Die folgende Auflistung zeigt detailliert, welche Themen Sie in der Woche behandeln und bearbeiten. Sie sind eine Voraussetzung für die folgenden Wochen und sollten gut verstanden worden sein. Wenn es Verständnisprobleme gibt, machen Sie sich Notizen und fragen Sie am Präsenztage nach, so dass wir gemeinsam zu Lösungen kommen können. Und denken Sie bitte immer daran: es gibt keine „dummen“ Fragen!

- Testen von Anwendungen:
 - Einführung in Softwaretests
 - Warum Tests wichtig sind
 - Unterschied zwischen manuellen und automatisierten Tests
 - Arten von Softwaretests (Unit Tests, Integration Tests, Funktionale Tests)
 - Testabdeckung und Testgetriebene Entwicklung (TDD)
- Unittest
 - Einführung in das unittest Modul
 - Testfall und TestSuite
 - Test Runner und Test Loader
 - Schreiben und Ausführen von Unit Tests
 - Verwendung von Assertions
- Dokumentation von Anwendungen:
 - Warum Dokumentation wichtig ist
 - Arten der Dokumentation (Code Kommentare, README, API Dokumentation)
 - Einführung in Python Docstrings
 - Verwendung von Sphinx zur Erstellung von Dokumentationen
 - Automatisierte Dokumentation mit Read the Docs

Lernpfad

Der Lernpfad ist ein Vorschlag, in welcher Reihenfolge Sie die Inhalte der Woche angehen können. Betrachten Sie ihn gerne als eine Todo-Liste, die Sie von oben nach unten abhaken. So können Sie sicher sein, dass Sie alle wichtigen Themen bearbeitet haben und sind gut vorbereitet für die folgenden Wochen.

1. Testen von Anwendungen:
 - Beschäftigen Sie sich mit der Wichtigkeit des Testens von Anwendungen.
 - Verstehen Sie, warum Tests für die Qualitätssicherung in der Softwareentwicklung unerlässlich sind.
 - Machen Sie sich mit den verschiedenen Testebenen vertraut, einschließlich UnitTests, Integrationstests, Funktionstests, Systemtests und Akzeptanztests.
 - Lernen Sie, wie Sie Tests für Ihre Anwendungen entwerfen und durchführen können.
2. Unittest:
 - Führen Sie sich in das unittest Framework ein, das in Python für das Schreiben von Unit-Tests verwendet wird.
 - Lernen Sie die Struktur des Frameworks kennen und wie Sie Testfälle und Test-Suites erstellen können.
3. Dokumentation von Anwendungen:
 - Erkunden Sie die Bedeutung einer guten Dokumentation für Ihre Anwendungen.
 - Lernen Sie, wie Sie Dokumentationen schreiben, die sowohl für Sie als auch für andere Entwickler nützlich sind.
 - Erfahren Sie mehr über die verschiedenen Formen der Dokumentation, einschließlich Code-Kommentare, Readme-Dateien und formelle Dokumentation.
4. Übungsaufgaben
 - Erstellen Sie Testfälle für Ihre bisherigen Anwendungen oder Übungen unter Verwendung von Unittest.
 - Entwerfen Sie einen Code, der testbar ist, indem Sie ihn in Funktionen und Module aufteilen.
 - Dokumentieren Sie eine Ihrer Anwendungen: Kommentieren Sie den Code, erstellen Sie eine Readme-Datei und eine formelle Dokumentation.
5. Selbstbewertung
6. Online-Schulungstag

Programmieraufgaben

Die folgenden Programmieraufgaben sollen Ihnen eine Anregung geben. Haben Sie eigene Ideen und Themen, die Sie ausprobieren wollen, dann sollten Sie diesen nachgehen. Wichtig ist vor allem, dass Sie „Dinge ausprobieren“. Und auch, dass Sie Fehler machen, sowohl syntaktische als auch semantische. Versuchen Sie diese Fehler zu finden und aufzulösen, dann gerade aus den Fehlern lernen Sie am Ende am meisten.

1. Reguläre Ausdrücke:

Schreiben Sie ein Python-Skript, das mithilfe von regulären Ausdrücken alle Email-Adressen aus einem gegebenen Text extrahiert.

2. Testen von Anwendungen mit Unittest:

Entwickeln Sie eine einfache Funktion, zum Beispiel eine Funktion, die die Fakultät einer Zahl berechnet. Schreiben Sie dann Testfälle für diese Funktion mithilfe des Unittest-Frameworks.

Abschluss-Quiz

Das Quiz soll Ihnen einen ersten Hinweis auf Ihren Lernfortschritt geben. Nach unserer Einschätzung sollten Sie diese Fragen alle beantworten können, wenn Sie den Stoff der Woche durchgearbeitet und verstanden haben. Natürlich gibt es noch sehr viel mehr mögliche Fragen, dazu wollen wir auf die Literatur und das Internet verweisen. Geben Sie gerne einmal „python quizzes“ bei Google ein.

1. Welche Methode wird in der Python-Standardbibliothek verwendet, um einen String zu durchsuchen, der ein Muster übereinstimmt?
 - a) `Re.match()`
 - b) `re.find()`
 - c) `re.search()`
 - d) `re.findall()`
2. Welche Methode wird in Unittest verwendet, um zu überprüfen, ob eine Bedingung wahr ist?
 - a) `assertTrue()`
 - b) `assertEqual()`
 - c) `assertIn()`
 - d) `assertIsNone()`
3. Welche Art von Tests wird verwendet, um sicherzustellen, dass mehrere Komponenten einer Anwendung zusammenarbeiten?
 - a) Unit-Tests
 - b) Integrationstests
 - c) Funktionstests
 - d) Regressionstests
4. Welche der folgenden Dokumentationsarten ist kein Teil einer guten Code-Dokumentation?
 - a) Code-Kommentare
 - b) Eine README-Datei
 - c) Eine detaillierte Nutzerdokumentation
 - d) Ein detaillierter Geschäftsplan
5. Welche Methode wird in Unittest verwendet, um auf Gleichheit zu prüfen?
 - a) `assertEqual()`
 - b) `assertTrue()`
 - c) `assertRaises()`
 - d) `assertIn()`